

SCRYFRAME

*Programa de auto etiquetación inteligente
de imágenes*



Jorge De Diego Jiménez

CFGs DAM

Curso 25-26



Progresa
Una formación de futuro

ÍNDICE

INTRODUCCIÓN.....	4
DESCRIPCIÓN.....	4
OBJETIVOS GENERAL.....	4
PLANIFICACIÓN	4
PRESUPUESTO.....	4
ANÁLISIS DE MERCADO.....	5
PÚBLICO OBJETIVO Y PERFILES DE USUARIO	5
COMPETENCIA.....	5
PROPUESTA.....	5
DEFINICIÓN DE OBJETIVOS/ESPECIFICACIONES DEL PRODUCTO	5
DISEÑO	6
DIAGRAMAS DE NAVEGACIÓN (USER FLOW).....	6
DISEÑOS GRÁFICO E INTERFACES.....	6
FIDELIDAD BAJA.....	6
FIDELIDAD MEDIA.....	6
FIDELIDAD ALTA.....	6
DESARROLLO	7
ARQUITECTURA GENERAL DE LA APLICACIÓN/SISTEMA/SERVICIO	7
HERRAMIENTAS DE DESARROLLO	7
LENGUAJES DE PROGRAMACIÓN, FRAMEWORKS Y LIBRERIAS UTILIZADAS.....	7
MODELO DE DATOS.....	7
INSTRUCCIONES DE DESPLIEGUE DE LA APLICACIÓN.....	7
CONCLUSIONES Y LÍNEAS DE FUTURO	8
CONCLUSIONES	8
LÍNEAS DE FUTURO	8

INTRODUCCIÓN

DESCRIPCIÓN

Scryframe es una aplicación web de catalogación de imágenes pensada para uso local. Su objetivo es ayudar a artistas, ilustradores y coleccionistas a organizar grandes colecciones personales mediante un sistema de etiquetas (tags) asistido por un modelo de aprendizaje automático.

El punto de partida del proyecto es la dificultad práctica de mantener etiquetadas miles de imágenes de forma manual. La propuesta de Scryframe es delegar esa tarea inicial a un modelo de Inteligencia Artificial entrenado para sugerir etiquetas descriptivas y dejar que el usuario únicamente confirme o ajuste la propuesta antes de guardar la imagen.

El modelo IA utiliza DeeepDanbooru, entrenado sobre el dataset público de Danbooru (una de las mayores bases de datos anotadas de ilustración digital), por lo que el modelo está especializado en arte e ilustración (estilo, personajes, composición, vestimenta, etc.) Conviene mencionar que el dataset original incluye material para adultos, por lo que el vocabulario aprendido contempla descriptores de esa naturaleza. Scryframe únicamente procesa las imágenes que el propio usuario sube voluntariamente a su instalación local, sin acceso a galerías externas, y la decisión sobre qué etiquetas guardar queda siempre en manos del usuario en el paso de confirmación.

OBJETIVOS GENERAL

Diseñar y desarrollar una aplicación que permita subir imágenes, obtener automáticamente una propuesta de etiquetas mediante un modelo IA, confirmarlas y guardarlas, y posteriormente consultar la colección filtrando por una o varias etiquetas combinadas.

PLANIFICACIÓN

- **Análisis e investigación:** 15 días
 - De los métodos de escaneo automático de imágenes y entrenamiento de modelos de forma local
- **Backend:** 20 días
 - La estructura hexagonal / Domain Driven Design actual constituye un cambio en el diseño del proyecto que requirió revisiones
- **Comprobaciones** 5 días
 - Ajustes finales y documentación

PRESUPUESTO

- Amortización contable (3 años): $4\,800\text{ €} / 3\text{ años} = 1\,600\text{ €/año}$ ($\approx 133\text{ €/mes}$), incorporada como gasto anual a la cuenta de resultados.
- Modelo de negocio propuesto (freemium): versión gratuita limitada y versión Premium a 3 €/mes con catálogo ilimitado y exportación de tags.
- Ingresos previstos (escenario base): $200\text{ usuarios premium} \times 3\text{ €/mes} = 600\text{ €/mes}$ ($7\,200\text{ €/año}$). Se asume una base de usuarios modesta (ilustradores, coleccionistas, creadores de contenido).
- Punto de equilibrio (break-even): $4\,800\text{ €} / 600\text{ €/mes} \approx 8\text{ meses}$.
- ROI primer año: $(7\,200\text{ €} - 4\,800\text{ €}) / 4\,800\text{ €} \approx 50\%$.
- Margen operativo 1.º año (descontando amortización): $7\,200\text{ €} - 1\,600\text{ €} = 5\,600\text{ €}$.

Estimaciones alternativas para sensibilidad:

- 1- Escenario conservador (100 usuarios premium): $3\,600\text{ €/año} \rightarrow \text{break-even} \approx 16\text{ meses}$, ROI 1.er año $\approx -25\%$.
- 2- Escenario optimista (400 usuarios premium): $14\,400\text{ €/año} \rightarrow \text{break-even} \approx 4\text{ meses}$, ROI 1.er año $\approx 200\%$.

ÁNALISIS DE MERCADO

PÚBLICO OBJETIVO Y PERFILES DE USUARIO

Scryframe está pensado para usuarios particulares que mantienen colecciones personales de imágenes en su propio ordenador, ya sean profesionales como ilustradores y diseñadores, como aficionados al arte digital y coleccionistas en general. Comparten el problema de tener miles de archivos sin organizar y la necesidad de poder localizar rápidamente piezas concretas mediante atributos visuales (personajes, estilos, composición, colores, etc.).

El perfil principal es por tanto un usuario técnico-medio, cómodo instalando una aplicación local en Windows, que valora el control sobre sus propios datos frente a soluciones en la nube.

COMPETENCIA

A continuación se analizan alternativas más representativas en el espacio de catalogación de imágenes con etiquetas, agrupando puntos fuertes y débiles en relación con la propuesta de Scryframe.

Hydrus Network. Aplicación local de código abierto orientada al archivo masivo de imágenes mediante un sistema de tags muy potente. Es probablemente el competidor más directo en cuanto a planteamiento (local-first, basado en etiquetas, sin dependencia de la nube).

- Fortalezas: ecosistema maduro, importación masiva, deduplicación por hash, sincronización opcional entre instalaciones, integraciones con scrapers.
- Debilidades: curva de aprendizaje muy pronunciada, interfaz densa heredada de Qt poco accesible para usuarios no técnicos y, sobre todo, **no incluye etiquetado automático asistido por IA**: cada tag debe asignarse manualmente o importarse desde fuentes externas.

Eagle. Gestor comercial de activos digitales (≈ 30 € licencia única) muy popular entre diseñadores e ilustradores.

- Fortalezas: experiencia de usuario muy pulida, vistas tipo Pinterest, soporte para múltiples formatos (PSD, AI, vídeo, fuentes), integración con Figma/Sketch, paletas y carpetas inteligentes.
- Debilidades: cerrado y de pago, sin etiquetado automático real (solo asigna etiquetas por carpeta o color predominante), enfocado al flujo profesional de diseño más que al archivo personal masivo.

Posicionamiento de Scryframe frente a la competencia. El hueco que ocupa Scryframe es el de un gestor local sencillo -entre la sofisticación abrumadora de Hydrus y el coste/cerrado de Eagle- que reduce drásticamente la fricción del etiquetado gracias a DeepDanbooru, mantiene los datos siempre en el equipo del usuario, y replica la UX de búsqueda por intersección de tags que ya es estándar en los sitios tipo Booru.

PROPUESTA

Scryframe combina un control archivos locales con la facilidad de búsqueda por tags junto un sistema de etiquetado automático asistido por IA. El usuario sube una imagen, revisa una propuesta de tags con estimaciones de la certeza del modelo y, con un solo clic, persiste la imagen en su catálogo fácil de filtrar.

DEFINICIÓN DE OBJETIVOS/ESPECIFICACIONES DEL PRODUCTO

Funcionalidades del Producto Mínimo Viable (PMV):

- Subir una imagen desde el navegador y previsualizarla.
- Obtener automáticamente una lista de etiquetas predichas por DeepDanbooru, ordenadas por confianza.
- Permitir al usuario confirmar, descartar o añadir etiquetas antes de guardar.
- Persistir la imagen en disco junto con los metadatos y las etiquetas confirmadas.
- Consultar el catálogo en una galería paginada ($4 \times 4 = 16$ imágenes por página).
- Filtrar la galería por una o varias etiquetas combinadas (intersección).

DISEÑO

DIAGRAMAS DE NAVEGACIÓN (USER FLOW)

El flujo de usuario de Scryframe se ha diseñado en torno a dos tareas principales -subir y catalogar imágenes, y consultar el catálogo- resueltas en dos pantallas distintas conectadas por una cabecera de navegación común.

1. PANTALLA INICIAL - UPLOAD

Al abrir la aplicación, el usuario es redirigido automáticamente a /upload. Esta pantalla concentra todo el flujo de incorporación de una nueva imagen al catálogo y se modela como una máquina de estados con seis estados claramente diferenciados: idle (sin imagen), selected (imagen elegida y previsualizada), analyzing (esperando al modelo), tagging (revisando etiquetas), saving (persistiendo) y saved (operación finalizada). Existe además un estado paralelo error que permite recuperar la operación.

Recorrido típico: el usuario hace clic en «Choose an image...», selecciona un archivo del disco y se le muestra una previsualización inmediata generada con `URL.createObjectURL`. Pulsa «Upload & analyze» y la aplicación envía el archivo al backend mediante `POST /images/upload`. Mientras se ejecuta DeepDanbooru se muestra un mensaje informativo, ya que el proceso puede tardar varios segundos. Cuando llega la respuesta, se renderiza la lista de etiquetas predichas ordenadas por confianza descendente; las que superan el umbral del 75% aparecen ya marcadas como sugerencia. El usuario puede marcar/desmarcar individualmente, usar los atajos «Select all» / «Select none» y, finalmente, pulsar «Save with N tags», que dispara `POST /images` con el mismo `imageId` que devolvió el upload, manteniendo así la trazabilidad entre el archivo guardado en disco y la entidad en base de datos. Tras el éxito, un mensaje «Saved!» y un botón «Upload another» permiten reiniciar el flujo.

2. PANTALLA SECUNDARIA - GALLERY

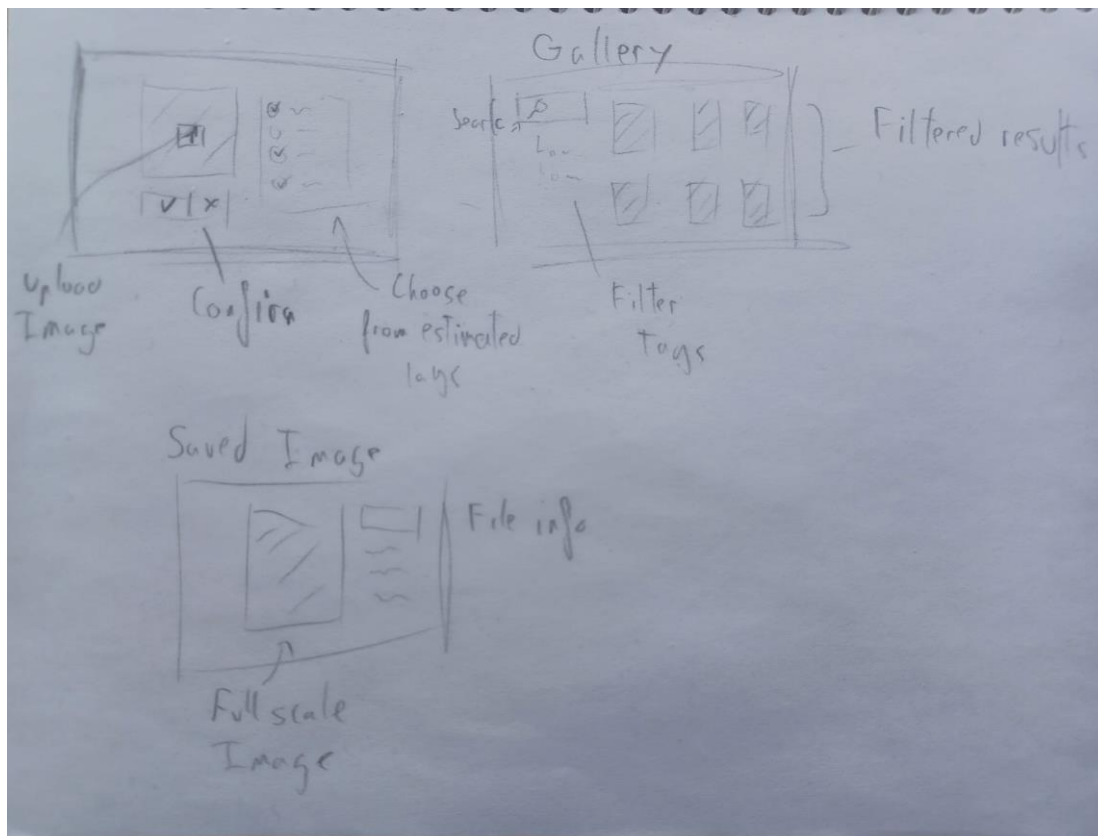
Accesible desde el enlace «Gallery» de la cabecera. Por defecto carga el catálogo completo (`GET /images/list`) y lo renderiza paginado. La barra superior funciona como buscador por chips: el usuario escribe un tag y pulsa Enter para añadirlo como chip; un retroceso en un input vacío elimina el último chip; el botón «Clear all» limpia todos. Cualquier modificación de los chips dispara automáticamente una nueva consulta -sin chips se llama a `GET /images/list`, y con uno o más se llama a `GET /images/search?tag=...&tag=...` con semántica de intersección-. Un botón ↻ permite refrescar la vista tras guardar nuevas imágenes.

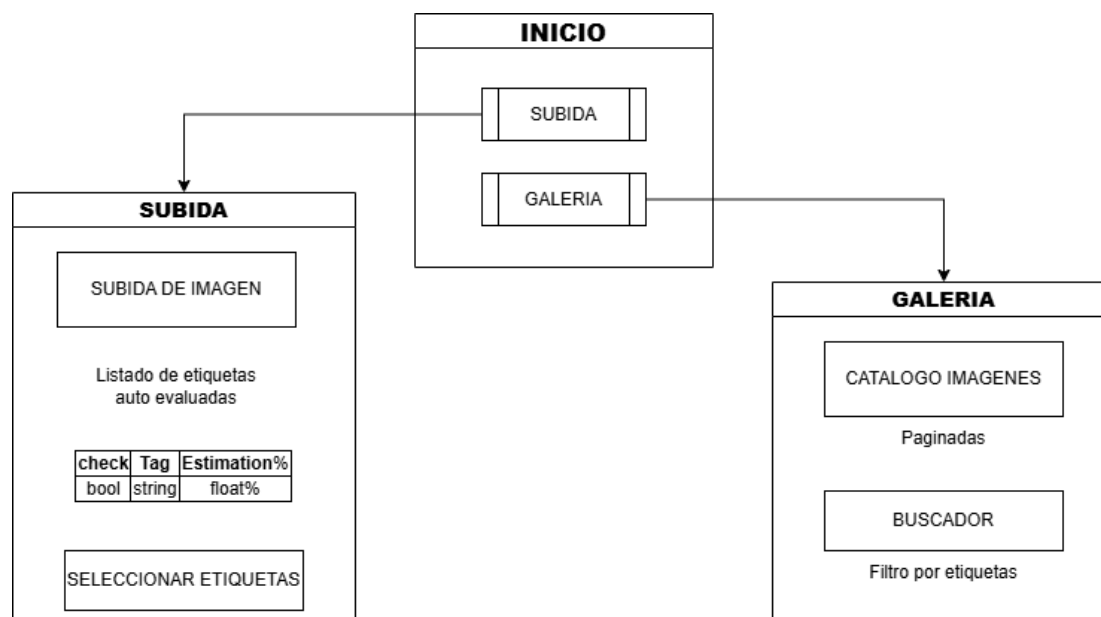
3. NAVEGACIÓN GENERAL

Una cabecera persistente con la marca «Scryfall» y los enlaces «Upload» / «Gallery» permite al usuario alternar entre ambas pantallas en cualquier momento. La ruta raíz («/») redirige a «/upload», y cualquier ruta no reconocida también redirige allí, garantizando que el usuario nunca llegue a una pantalla en blanco.

DISEÑOS GRÁFICO E INTERFACES

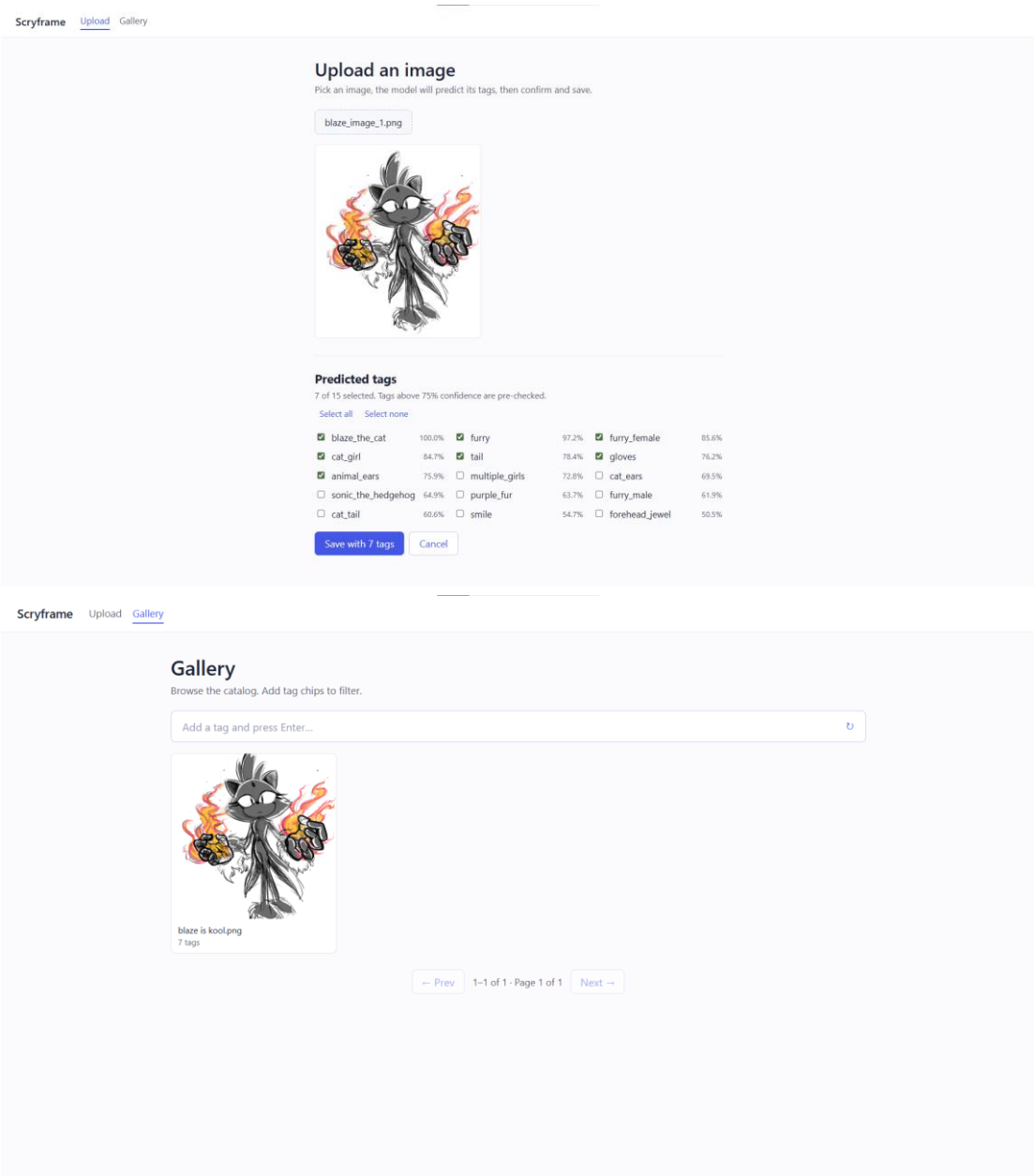
FIDELIDAD BAJA





FIDELIDAD ALTA

El prototipo de alta fidelidad coincide con la implementación final, ya que se desarrolló iterando directamente sobre componentes Angular reales.



DESARROLLO

ARQUITECTURA GENERAL DE LA APLICACIÓN/SISTEMA/SERVICIO

Scryframe sigue una arquitectura desacoplada compuesta por tres capas independientes que se comunican exclusivamente vía HTTP/REST sobre la red local del usuario.

FRONTEND - Aplicación web Angular con componentes standalone. Se ha optado por web (no nativo) porque el backend ejecuta un proceso local de Windows que requiere una máquina concreta, lo que descarta los escenarios móviles por el momento. La comunicación con el backend se realiza mediante HttpClient con la API Fetch.

BACKEND - API REST Spring Boot 4, siguiendo una arquitectura hexagonal / Domain-Driven Design con tres capas: domain (agregados Image y Tag, value objects, interfaces de repositorio), application (servicios y DTOs) e infrastructure (controladores REST, adaptadores JPA, almacenamiento en disco).

BASE DE DATOS - Utiliza JPA/Hibernate para el mapeo objeto-relacional. Las imágenes en sí (archivos binarios) NO se guardan en la base de datos sino en el sistema de archivos, en una carpeta configurable mediante la propiedad scryframe.storage.path; la tabla images solo guarda los metadatos y la relación N:N con tags a través de una tabla intermedia tagged_images.

COMPONENTE EXTERNO - DeepDanbooru, un modelo de visión por computador basado en una red ResNet personalizada, ejecutado mediante un subproceso cmd.exe que invoca el comando «deepdanbooru evaluate». La salida estándar del proceso se parsea con una expresión regular que captura las líneas de la forma «(0.998) tag_name» y las convierte en una lista de PredictedTagDTO.

HERRAMIENTAS DE DESARROLLO

IDEs: IntelliJ IDEA Community Java, geppara el backend y Webstorm para el frontend.

CONTROL DE VERSIONES: Git, con el frontend y el backend ubicados como subcarpetas hermanas dentro de un repositorio raíz.

HERRAMIENTAS DE LÍNEA DE COMANDOS: Angular CLI (ng) para scaffolding y servidor de desarrollo, npm como gestor de paquetes, y deepdanbooru como CLI Python ejecutable desde cmd.exe.

PRUEBAS MANUALES DE LA API: Postman durante el desarrollo del backend, antes de tener la SPA funcional.

LENGUAJES DE PROGRAMACIÓN, FRAMEWORKS Y LIBRERÍAS UTILIZADAS

BACKEND

- Lenguaje: Java 25 (con posibilidad de bajar a Java 21 LTS si la disponibilidad de la JDK 25 supone un problema en el entorno de despliegue y/o dependencias con Maven).
- Framework: Spring Boot 4.0.6.
- Starters utilizados:
 - spring-boot-starter-webmvc (controladores REST)
 - spring-boot-starter-data-jpa (ORM)
 - spring-boot-starter-validation (Jakarta Validation)
 - spring-boot-starter-actuator (endpoints de salud),
 - spring-boot-devtools (recarga en caliente).
- Persistencia: H2 Database como motor embebido en runtime.
- Boilerplate: Project Lombok para generar getters, setters y builders mediante anotaciones (@Value, @Builder, @AllArgsConstructor, @NoArgsConstructor, @Getter).

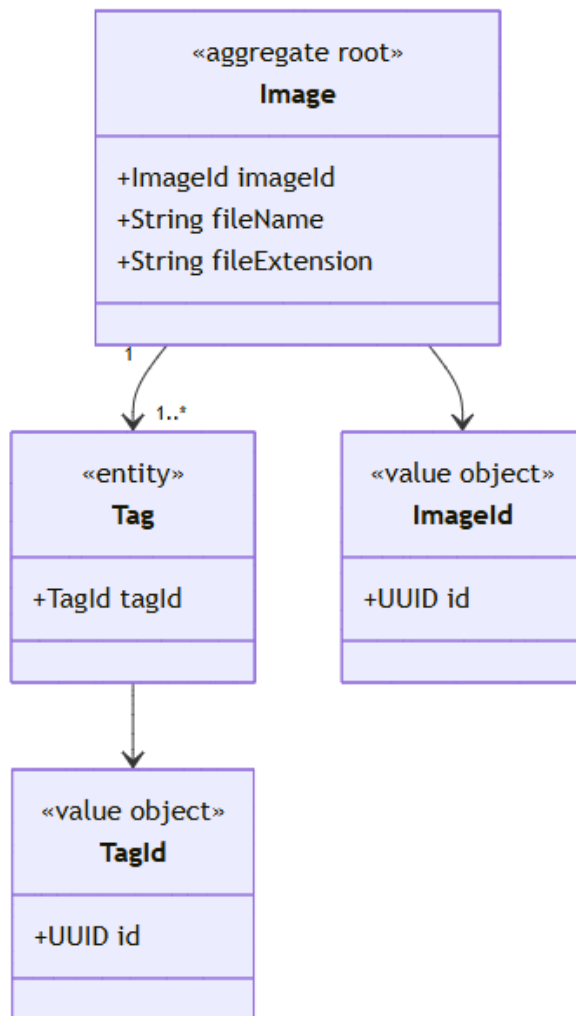
FRONTEND

- Lenguaje: TypeScript 5.9
- Framework: Angular 20.3
- Librerías Angular:
 - @angular/common/http (HttpClient)
 - @angular/router (enrutado)
 - @angular/forms (input controls).
- Reactividad: RxJS 7.8 (Observables devueltos por HttpClient), signals para estado local de componente.
- Estilos: SCSS con variables locales por componente (sin librerías de UI externas).

MODELO DE IA

- DeepDanbooru: red ResNet open-source personalizada entrenada sobre el dataset de Danbooru, ejecutada como CLI Python (TensorFlow) Repositorio en GitHub: <https://github.com/KichangKim/DeepDanbooru>
El proyecto requiere el modelo en formato Keras bajo la carpeta Scryfall_backend/model/. Si no se encuentra debe obtenerse o bien entrenando un propio modelo o descargando desde el enlace en el README del proyecto.

MODELO DE DATOS



```
{
  "id": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee",
  "fileName": "image_1",
  "extension": "jpg",
  "tags": [
    {
      "id": "tag_1"
    },
    {
      "id": "tag_2"
    }
  ]
}
```

INSTRUCCIONES DE DESPLIEGUE DE LA APLICACIÓN

Debido a que actualmente el modelo IA requiere invocarse a través de cmd.exe, debe ejecutarse en local sobre Window. Los pasos para levantar el entorno completo son:

REQUISITOS PREVIOS

- JDK 25 (o 21 si se ajusta java.version en pom.xml). Verificar con «java -version» y «javac -version».
- Node.js ≥ 22 y npm ≥ 10 . Verificar con «node -v» y «npm -v».
- Angular CLI: «npm install -g @angular/cli».
- Python 3.11 y DeepDanbooru clonado
- Modelo entrenado de DeepDanbooru .keras ubicado en Scryframe_backend/model/ (archivo disponible para descargar mediante enlace en el README)

PASO 1 - CONFIGURACIÓN

Editar Scryframe_backend/src/main/resources/application.properties y ajustar las rutas relativas si fuera necesario:

```
scryframe.storage.path=./storage  
  
scryframe.deepdanbooru.project-path=./model  
  
spring.servlet.multipart.max-file-size=20MB  
  
spring.servlet.multipart.max-request-size=20MB
```

PASO 2 - INSTALACIÓN DE DEEPPDANBOORU

Para poder ejecutar los comandos de DeepDanbooru, se debe instalar primero. Necesitarás tener específicamente una versión Python 3.11, una vez lo tengas clona el repositorio de DeepDanbooru en local, accede por cmd a la carpeta, que debería contener 'requirements.txt' y ejecuta:

```
py -3.11 -m pip install -r requirements.txt  
py -3.11 -m pip install .[tensorflow]
```

PASO 3 - BASE DE DATOS

No requiere instalación. H2 se inicializa en memoria al arrancar Spring Boot y JPA crea las tablas (images, tags, tagged_images) automáticamente a partir de las anotaciones de las entidades. Para entornos con persistencia entre arranques bastaría con cambiar la URL JDBC a modo fichero (jdbc:h2:file:./data/scryframe).

PASO 4 - ARRANCAR EL BACKEND

```
cd Scryframe_backend
```

```
.\mvnw.cmd clean spring-boot:run
```

El servidor queda escuchando en <http://localhost:8080>.

PASO 5 - ARRANCAR EL FRONTEND

En otra terminal:

```
cd Scryframe_frontend
```

```
npm install    (solo la primera vez)
```

```
npm start
```

Angular CLI sirve la SPA en <http://localhost:4200> con recarga en caliente.

PASO 6 - USO

Abrir el navegador en <http://localhost:4200>. La ruta raíz redirige a `/upload`, donde se puede subir una imagen, revisar las etiquetas predichas por DeepDanbooru y guardarla. Una vez guardada, en `/gallery` se puede consultar el catálogo y filtrar por etiquetas usando los chips de búsqueda.

CONCLUSIONES Y LÍNEAS DE FUTURO

CONCLUSIONES

Se cumple el PMV objetivo: subida de imagen, predicción automática de etiquetas, confirmación por parte del usuario, persistencia en disco y galería paginada filtrable por uno o varios tags. El sistema acopla un backend Spring Boot con arquitectura limpia a una SPA Angular moderna y un modelo de IA externo, demostrando que es viable integrar tres tecnologías muy distintas a través de una API REST bien definida.

Las principales dificultades encontradas durante el desarrollo y sus soluciones: configuración inicial del modelo IA usando DeepDanbooru: el proyecto se encontraba muy desactualizado, así como las herramientas dedicadas a entrenar los modelos, por lo que se ha requerido tiempo en arreglar errores de estas dependencias. Por otra parte, mantener el mismo imageId durante el flujo upload/save requería atención en el diseño del contrato API, y similarmente se ha necesitado un parseo robusto de la salida de DeepDanbooru a través de regex;

LÍNEAS DE FUTURO

Mejoras y ampliaciones identificadas para futuras iteraciones:

- Paginación en el servidor mediante Spring Data Pageable, necesaria si la colección crece a miles de imágenes.
- Generación de thumbnails para reducir el ancho de banda en la galería, accediendo a las imágenes en su resolución completa al acceder individualmente.
- Edición de etiquetas después de guardar (actualmente solo es posible al crear la imagen).
- Vista de detalle de imagen con sus tags clicables para realizar nuevas búsquedas.
- Persistencia en disco de la base de datos (cambiar H2 a modo fichero) o migración a PostgreSQL.
- Sustitución del proceso cmd.exe por una integración nativa con TensorFlow Java o por un microservicio Python expuesto como REST, eliminando así la dependencia exclusiva de Windows.
- Soporte para gestión de usuarios y catálogos múltiples si se quisiera convertir en un servicio multi-usuario.

